

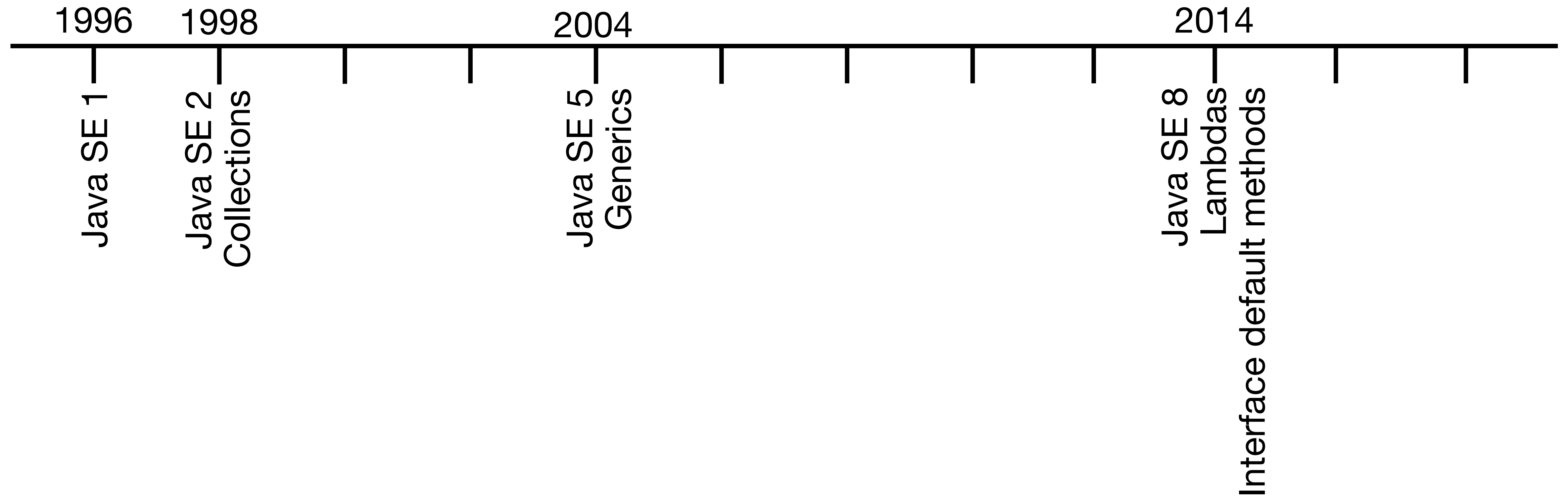
Collections

Java Advanced Developer

StayAhead Training; Aug, 2025

About Collections

Collections



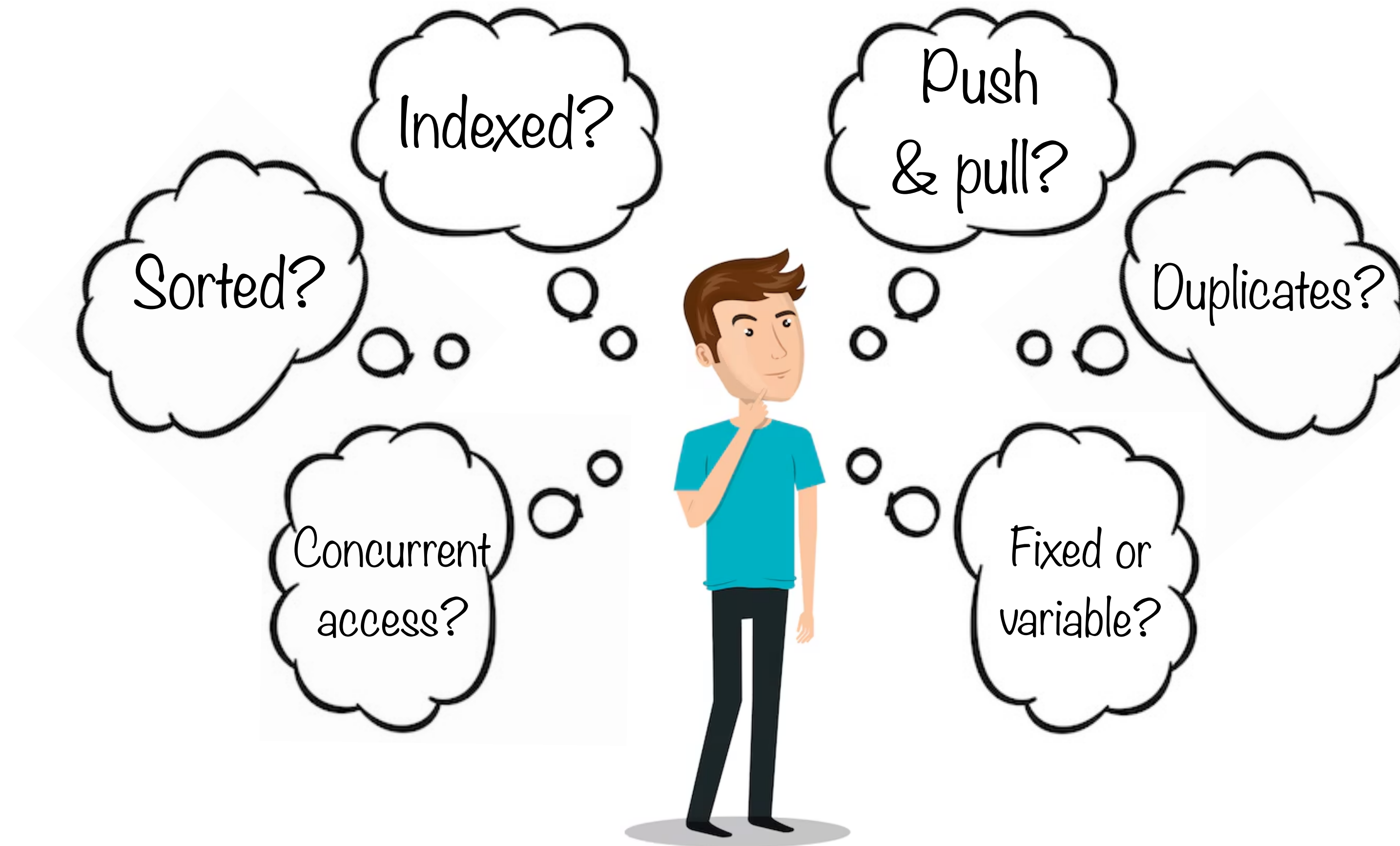
Collections Framework

Collections

- Set of interfaces to model various ways of storing data
- At least one implementation of each interface
- `java.util.*`

Collections Framework

Collections



Collections Framework

Collections

- Two main categories of interface:
 1. Collection: object storage and iteration*
 2. Map: key value pairs
- *Queues & Stacks are Collections but not concerned with iteration
- All Collections implement Iterable (see also Iterator)

Collections Framework

Collections

Pre Java SE 2 Collection	Modern Equivalent
Vector	ArrayList
Stack	ArrayDeque
HashTable	HashMap

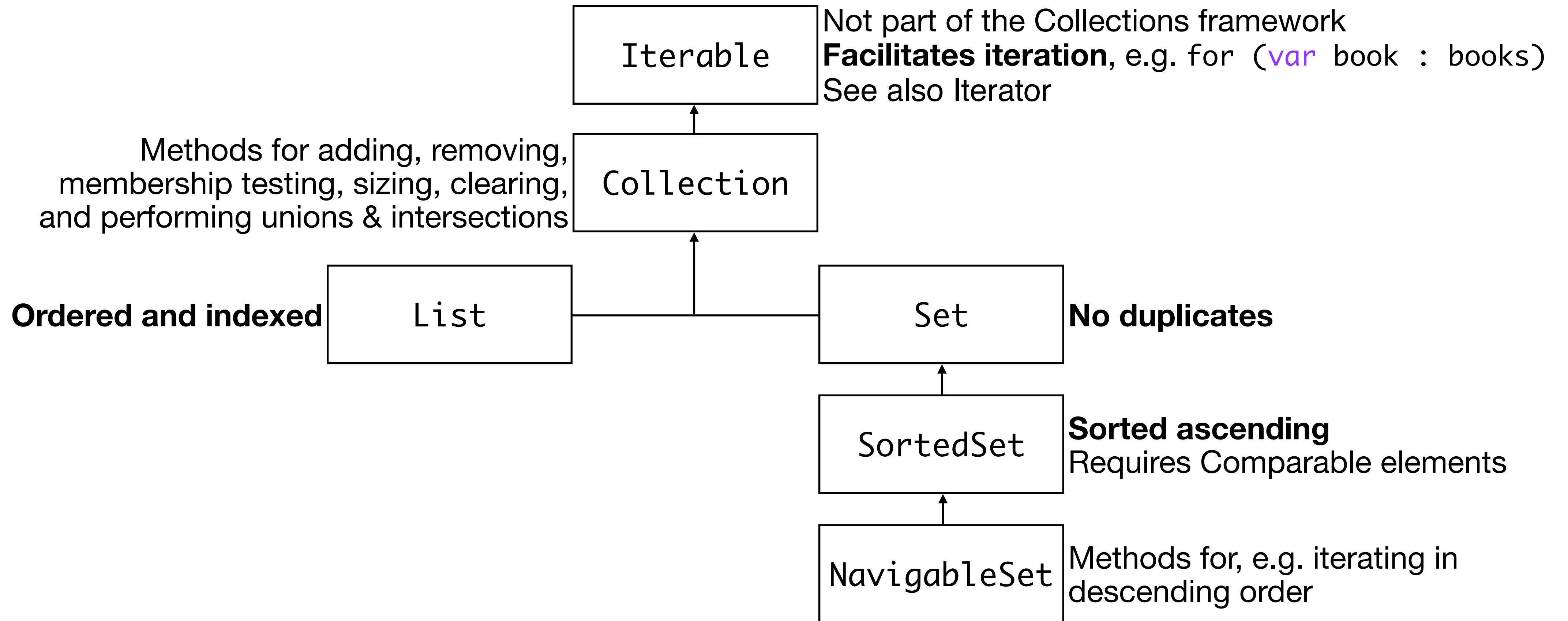
Collection vs Array

Collections

- Collections track the number of elements they contain
- Collections are **not** fixed size
- Collections can be queried to test for the presence of an element
- Collections provide functionality like intersecting and merging

Collection Hierarchy

Collections



Collection Methods

Collections

```
Collection<Integer> nums = new ArrayList<>();
```

```
boolean changed = nums.add(1);
```

```
// changed ==> true
```

```
boolean changed = nums.remove(1);
```

```
// changed ==> true
```

```
boolean contains1 = nums.contains(1);
```

```
// contains1 ==> false
```

Collection Methods

Collections

```
Collection<Integer> nums = new ArrayList<>();
```

```
boolean changed = nums.addAll(List.of(1, 2, 3, 4, 5));
```

```
// changed ==> true
```

```
// nums ==> [1, 2, 3, 4, 5]
```

```
boolean changed = nums.removeAll(List.of(3, 4, 5));
```

```
// changed ==> true
```

```
// nums ==> [1, 2]
```

Collection Methods

Collections

```
Collection<Integer> nums1 = new ArrayList<>();  
Collection<Integer> nums2 = new ArrayList<>();  
nums1.addAll(List.of(1, 2, 3));  
nums2.addAll(List.of(3, 4, 5));
```

```
boolean containsAll = nums1.containsAll(nums2);  
// containsAll ==> false
```

```
boolean changed = nums1.retainAll(nums2);  
// changed ==> true  
// nums1 ==> [3]
```


Collection Methods

Collections

```
Collection<Integer> nums = new ArrayList<>();  
nums.addAll(List.of(1, 2, 3));
```

```
int numElements = nums.size();  
// numElements ==> 3
```

```
boolean numsEmpty = nums.isEmpty();  
// numsEmpty ==> false
```

```
nums.clear();  
// nums ==> []
```

Collection Methods

Collections

```
Collection<Integer> nums = new ArrayList<>();  
nums.addAll(List.of(1, 2, 3));
```

```
Object[] objects = nums.toArray();
```



```
Integer[] integers = nums.toArray(new Integer[]{});
```

Iterating over a Collection

Collections

- Using the for-each pattern:

```
for (var element : collection) {  
    // process element  
}
```


Iterating over a Collection

Collections

- Using an Iterator:

```
var iterator = collection.iterator();  
  
while (iterator.hasNext()) {  
    var element = iterator.next();  
    // process element  
}
```

Collection Implementations

Collections

- No direct implementation of the Collection interface
- List **extends** Collection
- ArrayList **implements** List

Lists

Collections

- List **extends** Collection
- Elements maintained in the **order** in which they're added
- Elements are **indexed** (from 0, obvs)

List Methods

Collections

```
List<Integer> nums = new ArrayList<>();  
nums.addAll(List.of(2, 7, 4));
```

```
nums.add(0, 1);  
// nums ==> [1, 2, 7, 4]
```



The first arg. is the index

```
int replacedNum = nums.set(3, 3);  
// replacedNum ==> 4  
// nums ==> [1, 2, 7, 3]
```

List Methods

Collections

```
List<Integer> nums = new ArrayList<>();  
nums.addAll(List.of(1, 2, 7, 3));
```

```
int removedNum = nums.remove(2);  
// removedNum ==> 7  
// nums ==> [1, 2, 3]
```

```
int numAtIndex1 = nums.get(1);  
// numAtIndex1 ==> 2
```


List Methods

Collections

```
List<Integer> nums = new ArrayList<>();  
nums.addAll(List.of(1, 2, 3));
```

```
int numAtIndex3 = nums.get(3);  
// ==> IndexOutOfBoundsException
```

List Methods

Collections

```
List<Integer> nums = new ArrayList<>();  
nums.addAll(List.of(1, 2, 3));
```

```
int indexOf2 = nums.indexOf(2);  
// indexOf2 ==> 1
```

```
int lastIndexOf4 = nums.lastIndexOf(4);  
// lastIndexOf4 ==> -1
```

List Methods

Collections

```
List<Integer> nums = new ArrayList<>();  
nums.addAll(List.of(1, 2, 3, 4, 5));
```

```
List<Integer> view = nums.subList(1, 4);  
// view ==> [2, 3, 4]
```

Ending index is exclusive!

```
view.clear();  
// nums ==> [1, 5]
```

Whatever changes you make to the view will affect the original List!

List Methods

Collections

```
List<Integer> nums1 = new ArrayList<>();  
List<Integer> nums2 = new ArrayList<>();  
nums1.addAll(List.of(1, 5, 6));  
nums2.addAll(List.of(2, 3, 4));
```

```
boolean changed = nums1.addAll(1, nums2);  
// changed ==> true  
// nums1 ==> [1, 2, 3, 4, 5, 6]
```

List Methods

Collections

```
List<Integer> nums = new ArrayList<>();  
nums.addAll(List.of(2, 1, 3));
```

```
nums.sort(null);  
// nums ==> [1, 2, 3]
```

```
nums.sort(Comparator.naturalOrder());  
// nums ==> [1, 2, 3]
```


Iterating over a List

Collections

- `ListIterator` is a sub-type of `Iterator` and provides for:
 - specifying the index at which to start iterating
 - iterating in descending order
 - modifications whilst iterating

Iterating over a List

Collections

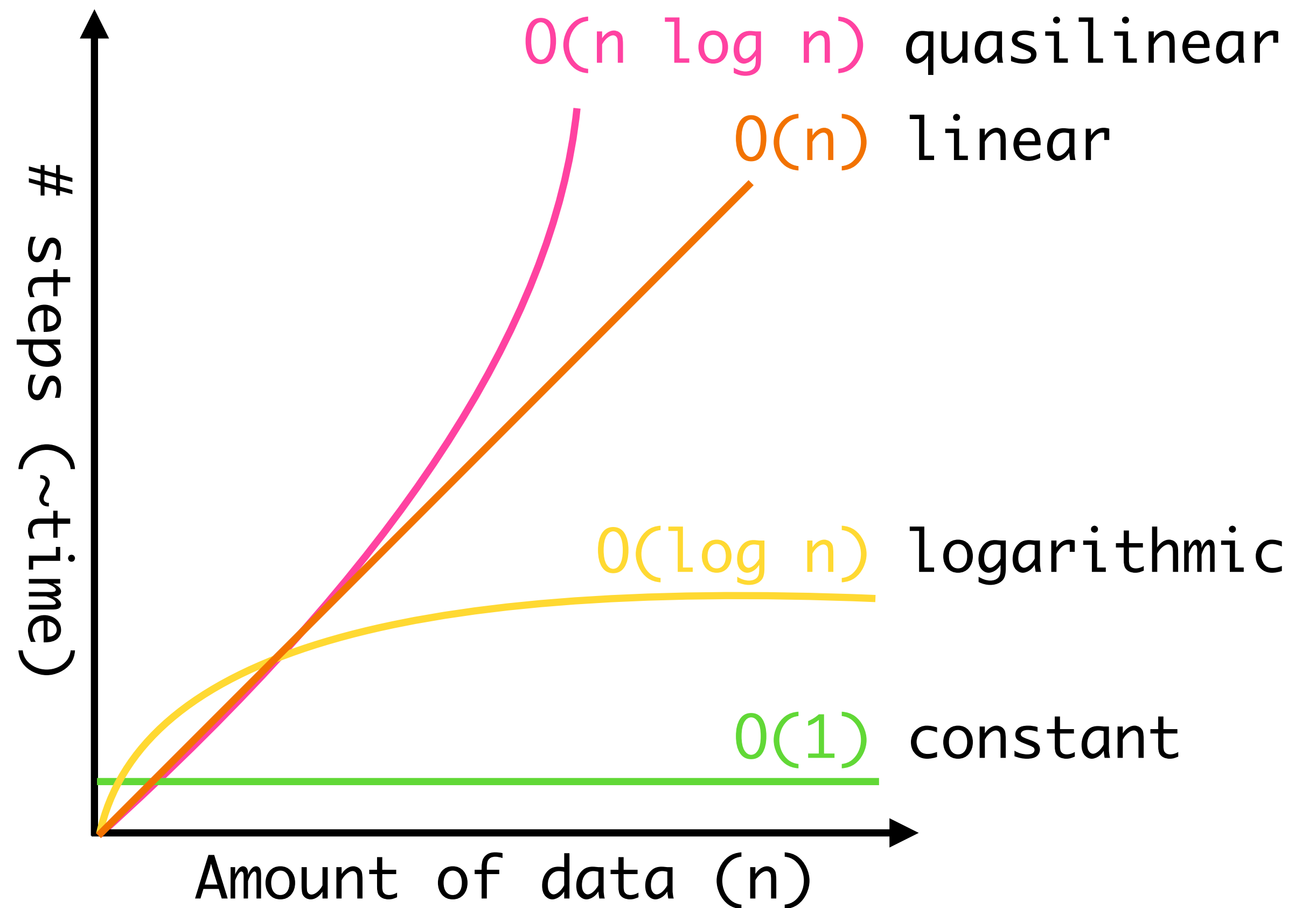
```
List<Integer> nums = new ArrayList<>();  
nums.addAll(List.of(1, 10, 3));
```

```
ListIterator<Integer> iterator = nums.listIterator(2);  
while (iterator.hasPrevious()) {  
    int num = iterator.previous();  
    if (num == 10) {  
        iterator.set(2);  
    }  
}  
// nums ==> [1, 2, 3]
```

Big O Notation

Collections

- How code slows as data grows
- To describe algorithm complexity
- O for ordnung (German) meaning order of approximation
- Constant best; linear OK



List Implementations

Collections

Operation	ArrayList	LinkedList
Reading the first element	$O(1)$	$O(1)$
Reading the last element	$O(1)$	$O(1)$
Reading an element in the middle	$O(1)$	$O(n)$
Adding an element at the end	$O(1)$	$O(1)$
Inserting an element at the start	$O(n)$	$O(1)$
Inserting an element in the middle	$O(n)$	$O(n)$

Sets

Collections

- Set **extends** Collection
- Elements maintained in **no particular order**
- Elements must be **unique** (no duplicates)
- No new methods
- One implementation: HashSet

Sets

Collections

```
Set<String> set = new HashSet<>();  
set.addAll(List.of("Tom", "Dick", "Tom", "Harry"));
```

```
for (var element : set) {  
    System.out.println(element);  
}
```

```
// output: Tom\nHarry\nDick
```


SortedSet

Collections

- SortedSet **extends** Set
- Elements sorted according to some comparison logic
- One implementation: TreeSet
- Elements must be Comparable or you must provide a Comparator

SortedSet

Collections

```
SortedSet<String> set = new TreeSet<>();  
set.addAll(List.of("Tom", "Dick", "Tom", "Harry"));
```

```
for (var element : set) {  
    System.out.println(element);  
}
```

```
// output: Dick\nHarry\nTom
```


SortedSet Methods

Collections

```
SortedSet<String> set = new TreeSet<>();  
set.addAll(List.of("Tom", "Dick", "Tom", "Harry"));
```

```
String first = set.first();  
// first ==> Dick
```

```
String last = set.last();  
// last ==> Tom
```

SortedSet Methods

Collections

```
SortedSet<Integer> set = new TreeSet<>();  
set.addAll(List.of(1, 3, 5, 2, 4, 6));
```

```
SortedSet<Integer> first3View = set.headSet(4);  
// first3View ==> [1, 2, 3]
```



toElement is exclusive!

```
SortedSet<Integer> last3View = set.tailSet(4);  
// last3View ==> [4, 5, 6]
```



fromElement is inclusive!

SortedSet Methods

Collections

```
SortedSet<Integer> set = new TreeSet<>();  
set.addAll(List.of(1, 3, 5, 2, 4, 6));
```

```
SortedSet<Integer> middle4View = set.subSet(2, 6);  
// middle4View ==> [2, 3, 4, 5]
```


NavigableSet

Collections

- NavigableSet **extends** SortedSet
- More methods
- Provides for iterating in descending order
- TreeSet retrofitted to implement NavigableSet

NavigableSet Methods

Collections

```
NavigableSet<Integer> set = new TreeSet<>();  
set.addAll(List.of(1, 5, 9, 2, 6, 10));
```

```
int smallestGreaterThanOrEqualTo3 = set.ceiling(3);  
// smallestGreaterThanOrEqualTo3 ==> 5
```

```
int biggestLessThanOrEqualTo8 = set.floor(8);  
// biggestLessThanOrEqualTo8 ==> 6
```

NavigableSet Methods

Collections

```
NavigableSet<Integer> set = new TreeSet<>();  
set.addAll(List.of(1, 5, 9, 2, 6, 10));
```

```
int smallestGreaterThan5 = set.higher(5);  
// smallestGreaterThan5 ==> 6
```

```
int biggestLessThan10 = set.lower(10);  
// biggestLessThan10 ==> 9
```


NavigableSet Methods

Collections

```
NavigableSet<Integer> set = new TreeSet<>();  
set.addAll(List.of(1, 5, 9, 2, 6, 10));
```

```
int first = set.pollFirst();  
// first ==> 1  
// set ==> [2, 5, 6, 9, 10]
```

```
int last = set.pollLast();  
// last ==> 10  
// set ==> [2, 5, 6, 9]
```

Iterating over a NavigableSet

Collections

```
NavigableSet<Integer> set = new TreeSet<>();  
set.addAll(List.of(1, 5, 9, 2, 6, 10));
```

```
Iterator<Integer> iterator = set.descendingIterator();  
while (iterator.hasNext()) {  
    int num = iterator.next();  
    System.out.println(num);  
}
```

```
// output: 10\n9\n6\n5\n2\n1
```


Immutable Collections

Collections

- List & Set have static factory methods for creating immutable collections:

```
List<Integer> nums = List.of(1, 1, 2, 2, 3, 3);  
Set<String> names = Set.of("Tom", "Dick", "Harry");  
Set<Integer> uniqueNums = Set.copyOf(nums);
```

- The implementation instance returned:
 - is **not** an ArrayList/HashSet
 - cannot be modified
 - will throw an exception if passed `null` (both) or duplicate values (Set only)

The Collections Class

Collections

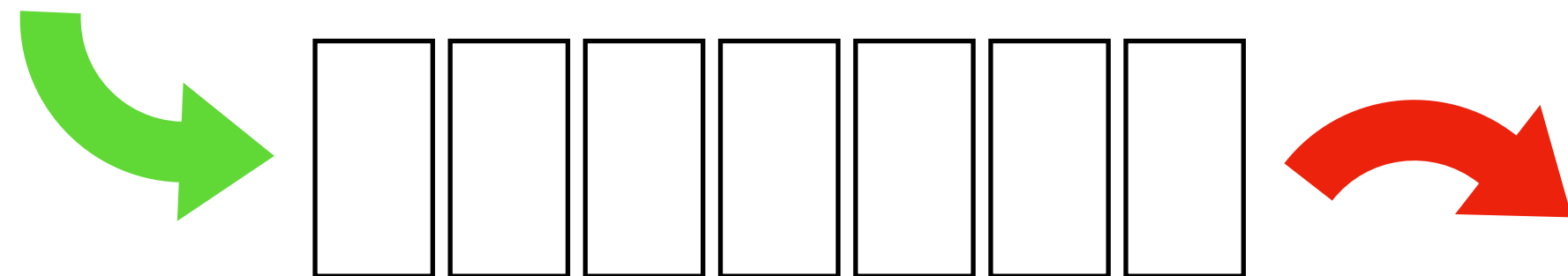
- The Collections (plural) class has many handy static methods, e.g.:

```
var nums = new ArrayList<>(List.of(1, 2, 3, 4, 5));  
var immutableNums = Collections.unmodifiableList(nums);  
var smallest      = Collections.min(nums);  
var biggest       = Collections.max(nums);  
                  Collections.shuffle(nums);  
var index0f3      = Collections.binarySearch(3);  
                  Collections.sort(nums);  
... 
```

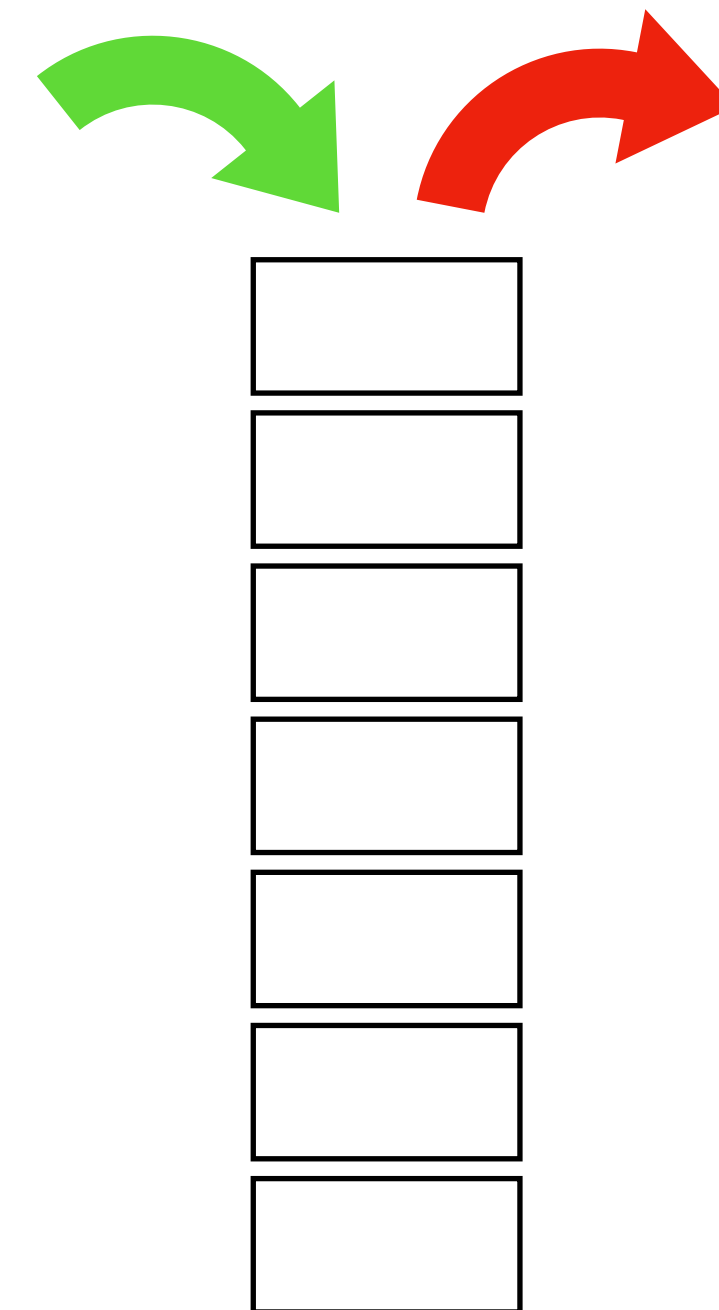
Assumes Comparable objects

Queues & Stacks

Collections



Queue - FIFO/LILO



Stack - LIFO/FILO

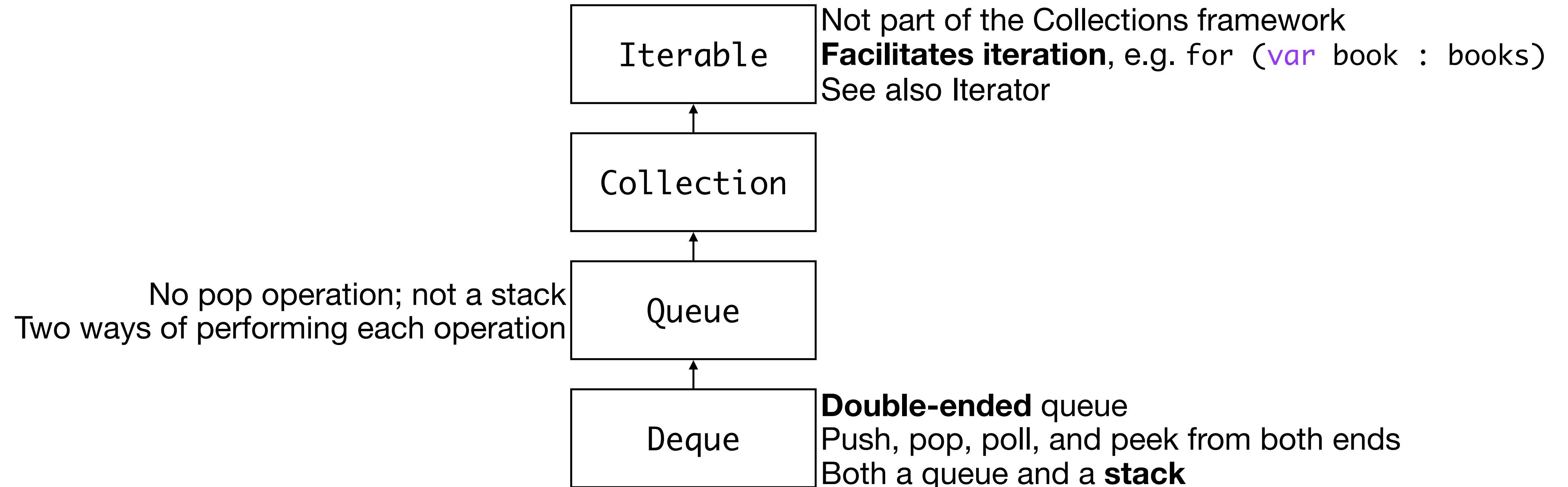
Queues & Stacks

Collections

Operation	Applies to	Description
Push	Both	Adds an element
Pop	Stack	Removes the newest element
Poll	Queue	Removes the oldest element
Peek	Both	Get the element to be removed on next call to pop/poll

Queue & Stack Hierarchy

Collections



Queue Methods

Collections

```
Queue<String> queue = new ArrayDeque<>();
```

```
boolean success = queue.add("Tom");  
// success ==> true  
// queue ==> [Tom]
```

Throws IllegalStateException
if the element can't be added

```
boolean success = queue.offer("Dick");  
// success ==> true  
// queue ==> [Tom, Dick]
```

Returns false if the
element can't be added

Queue Methods

Collections

```
Queue<String> queue = new ArrayDeque<>();  
queue.addAll(List.of("Tom", "Dick", "Harry"));
```

```
String oldest = queue.remove();  
// oldest ==> Tom  
// queue ==> [Dick, Harry]
```

Throws NoSuchElementException
if the queue is empty

```
String oldest = queue.poll();  
// oldest ==> Dick  
// queue ==> [Harry]
```

Returns null if the
queue is empty

Queue Methods

Collections

```
Queue<String> queue = new ArrayDeque<>();  
queue.addAll(List.of("Tom", "Dick", "Harry"));
```

```
String oldest = queue.element();
```

```
// oldest ==> Tom
```

```
// queue ==> [Tom, Dick, Harry]
```

Throws NoSuchElementException
if the queue is empty

```
String oldest = queue.peek();
```

```
// oldest ==> Tom
```

```
// queue ==> [Tom, Dick, Harry]
```

Returns null if the
queue is empty

Deque Methods

Collections

```
Deque<String> stack = new ArrayDeque<>();
```

```
stack.addFirst("Tom");  
// stack ==> [Tom]
```

Throws IllegalStateException
if the element can't be added

```
boolean success = queue.offerFirst("Dick");  
// success ==> true  
// stack ==> [Dick, Tom]
```

Returns false if the
element can't be added

Deque Methods

Collections

```
Deque<String> stack = new ArrayDeque<>();  
stack.addAll(List.of("Dick", "Tom"));
```

```
String latest = stack.removeFirst();  
// latest ==> Dick  
// stack ==> [Tom]
```

Throws NoSuchElementException
if the stack is empty

```
String latest = queue.pollFirst();  
// latest ==> Tom  
// stack ==> []
```

Returns null if the
stack is empty

Deque Methods

Collections

```
Deque<String> stack = new ArrayDeque<>();  
stack.addAll(List.of("Dick", "Tom"));
```

```
String latest = stack.getFirst();  
// latest ==> Dick  
// stack ==> [Dick, Tom]
```

Throws NoSuchElementException
if the stack is empty

```
String latest = queue.peekFirst();  
// latest ==> Dick  
// stack ==> [Dick, Tom]
```

Returns null if the
stack is empty

Deque Methods

Collections

- Note that Deque is both stack *and* queue
- It has methods to add, remove, & get from the tail of the collection too:
 - addLast
 - removeLast
 - getLast
- Confusingly, it also has push, pop, poll & peek methods, equivalent to:
 - addFirst
 - removeFirst
 - pollFirst
 - peekFirst

Queue & Deque Implementations

Collections

- `ArrayDeque` implements both and is backed by an array
- `LinkedList` implements both
- `PriorityQueue` implements Queue **only** and:
 - is backed by an array
 - keeps its elements **sorted** (via natural order or `Comparator`)
 - its **head is always the smallest**/least element

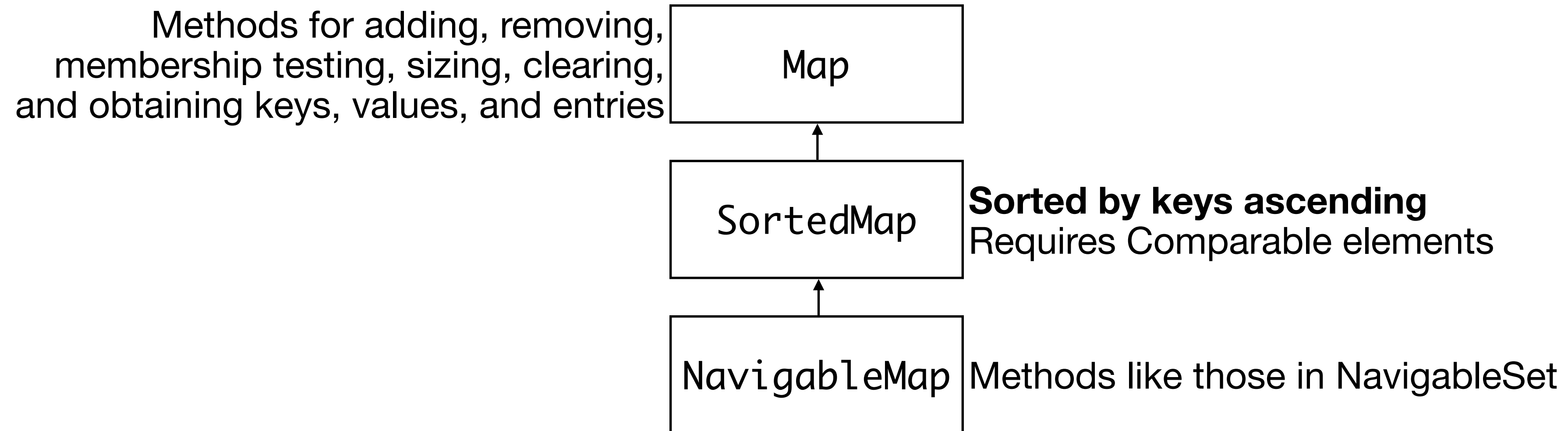
Maps

Collections

- Map does not extend Collection
- Key value pairs
- Keys must be simple, unique, hashable, and (ideally) immutable
- Values may be complex and duplicated
- Primary implementation: HashMap

Map Hierarchy

Collections



Storing Key Value Pairs in a Map Collections

- A key is bound to **one** value
- A value may be bound to **many** keys
- The set of keys cannot have duplicates so is a Set
- The set of value can have duplicates so is a Collection

Map Methods

Collections

```
Map<String, Integer> map = new HashMap<>();
```

```
Integer previousValue = map.put("Tom", 2);
```

```
// previousValue ==> null
```

```
// map ==> {Tom=2}
```

```
previousValue = map.put("Tom", 4);
```

```
// previousValue ==> 2
```

```
// map ==> {Tom=4}
```

Map Methods

Collections

```
Map<String, Integer> map = new HashMap<>();
```

```
Integer previousValue = map.putIfAbsent("Tom", 2);  
// previousValue ==> null  
// map ==> {Tom=2}
```

```
previousValue = map.putIfAbsent("Tom", 4);  
// previousValue ==> 2  
// map ==> {Tom=2}
```

Will succeed if value
assoc. with key is null

Map Methods

Collections

```
Map<String, Integer> map = new HashMap<>(Map.of(
    "Tom", 71, "Dick", 5, "Harry", 29
));
```

```
Integer tomsScore = map.get("Tom");
// tomsScore ==> 71
```

```
Integer billsScore = map.getOrDefault("Bill", -1);
// billsScore ==> -1
```

Would return
value assoc.
with key
if it existed

Map Methods

Collections

```
Map<String, Integer> map = new HashMap<>(Map.of(
    "Tom", 71, "Dick", 5, "Harry", 29
));
```

```
Integer tomsScore = map.remove("Tom");
// tomsScore ==> 71
```

```
Integer billsScore = map.remove("Bill");
// billsScore ==> null
```

Map Methods

Collections

```
Map<String, Integer> map = new HashMap<>(Map.of(
    "Tom", 71, "Dick", 5, "Harry", 29
));
```

```
boolean tomRemoved = map.remove("Tom", 71);
// tomRemoved ==> true
```

```
boolean billRemoved = map.remove("Bill", 33);
// billRemoved ==> false
```

Map Methods

Collections

```
Map<String, Integer> map = new HashMap<>(Map.of(
    "Tom", 71, "Dick", 5, "Harry", 29
));
```

```
boolean keyTomExists = map.containsKey("Tom");
// keyTomExists ==> true
```

```
boolean value29Exists = map.containsValue(29);
// value29Exists ==> true
```


Map Methods

Collections

```
Map<String, Integer> map = new HashMap<>(Map.of(
    "Tom", 71, "Dick", 5, "Harry", 29
));
```

```
Set<String> keyView = map.keySet();
// keyView ==> [Tom, Dick, Harry]
```

```
keyView.remove("Dick");
// map ==> {Tom=71, Harry=29}
```

Whatever changes you make
to the view will affect
the original Map!

Map Methods

Collections

```
Map<String, Integer> map = new HashMap<>(Map.of(
    "Tom", 71, "Dick", 5, "Harry", 29
));
```

```
var entriesView = map.entrySet();
// entriesView ==> [Tom=71, Dick=5, Harry=29]
```

```
Collection<Integer> valuesView = map.values();
// valuesView ==> [71, 5, 29]
```


SortedMap

Collections

- SortedMap **extends** Map
- Pairs sorted according to some comparison logic
- One implementation: TreeMap
- Keys must be Comparable or you must provide a Comparator

SortedMap

Collections

```
SortedMap<String, Integer> map = new TreeMap<>(Map.of(
    "Tom", 71, "Dick", 5, "Harry", 29
));
```

```
for (var entry : map.entrySet()) {
    System.out.println(entry.getKey());
}
```

```
// output: Dick\Harry\nTom
```

SortedMap Methods

Collections

```
SortedMap<String, Integer> map = new TreeMap<>(Map.of(
    "Tom", 71, "Dick", 5, "Harry", 29
));
```

```
String firstKey = map.firstKey();
// firstKey ==> Dick
```

```
String lastKey = map.lastKey();
// lastKey ==> Tom
```


SortedMap Methods

Collections

```
SortedMap<String, Integer> map = new TreeMap<>(Map.of(
    "Tom", 71, "Dick", 5, "Harry", 29
));
```

```
var toHarryView = map.headMap("Harry");  
// beforeHarryView ==> {Dick=5}
```

toKey is exclusive!

```
var fromHarryView = map.tailMap("Harry");  
// harryAndAfterView ==> {Harry=29, Tom=71}
```

fromKey is inclusive!

SortedMap Methods

Collections

```
SortedMap<String, Integer> map = new TreeMap<>(Map.of(
    "Tom", 71, "Dick", 5, "Harry", 29
));
```

```
var fromDickToTomView = map.subMap("Dick", "Tom");
// fromDickToTomView ==> {Dick=5, Harry=29}
```

NavigableMap

Collections

- NavigableMap **extends** SortedMap
- More methods
- Provides for traversing the map in descending order
- TreeMap retrofitted to implement NavigableMap

NavigableMap Methods

Collections

```
NavigableMap<Integer, String> map = new TreeMap<>(Map.of(
    1, "un", 3, "trois", 5, "cinq", 7, "sept", 9, "neuf"
));
```

```
Map.Entry<String, Integer> firstEntry = map.firstEntry();
// firstEntry ==> 1=un
```

```
Map.Entry<String, Integer> lastEntry = map.lastEntry();
// lastEntry ==> 9=neuf
```

NavigableMap Methods

Collections

```
NavigableMap<Integer, String> map = new TreeMap<>(Map.of(
    1, "un", 3, "trois", 5, "cinq", 7, "sept", 9, "neuf"
));
```

```
int smallestKeyGreaterThanOrEqualTo3 = map.ceilingKey(3);
// smallestKeyGreaterThanOrEqualTo3 ==> 3
```

```
int biggestKeyLessThanOrEqualTo8 = map.floorKey(8);
// biggestLessThanOrEqualTo8 ==> 7
```

```
// see also ceilingEntry and floorEntry
```


NavigableMap Methods

Collections

```
NavigableMap<Integer, String> map = new TreeMap<>(Map.of(
    1, "un", 3, "trois", 5, "cinq", 7, "sept", 9, "neuf"
));
```

```
int smallestKeyGreaterThan5 = map.higherKey(5);
// smallestKeyGreaterThan5 ==> 7
```

```
int biggestKeyLessThan10 = map.lowerKey(10);
// biggestKeyLessThan10 ==> 9
```

```
// see also higherEntry and lowerEntry
```

NavigableMap Methods

Collections

```
NavigableMap<Integer, String> map = new TreeMap<>(Map.of(
    1, "un", 3, "trois", 5, "cinq", 7, "sept", 9, "neuf"
));
```

```
var firstEntry = map.pollFirstEntry();
// firstEntry ==> 1=un
// map ==> {3=trois, 5=cinq, 7=sept, 9=neuf}
```

```
var lastEntry = map.pollLastEntry();
// lastEntry ==> 9=neuf
// map ==> {3=trois, 5=cinq, 7=sept}
```

Iterating over a NavigableMap

Collections

```
NavigableMap<Integer, String> map = new TreeMap<>(Map.of(
    1, "un", 3, "trois", 5, "cinq", 7, "sept", 9, "neuf"
));
```

```
var reversedMap = map.descendingMap();
for (var entry : reversedMap.entrySet()) {
    System.out.println(entry);
}
```

```
// output: 9=neuf\n7=sept\n5=cinq\n3=trois\n1=un
```

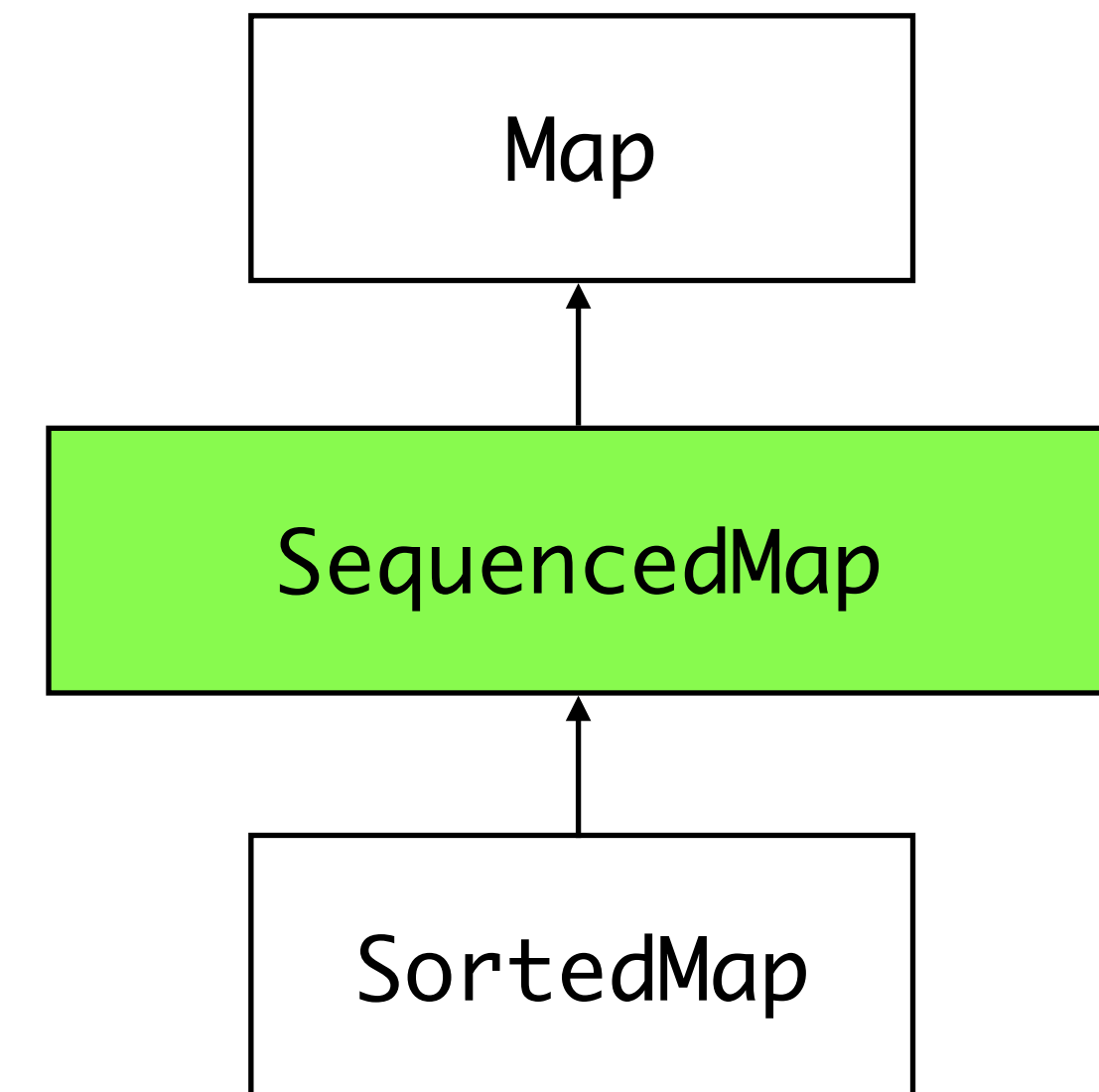
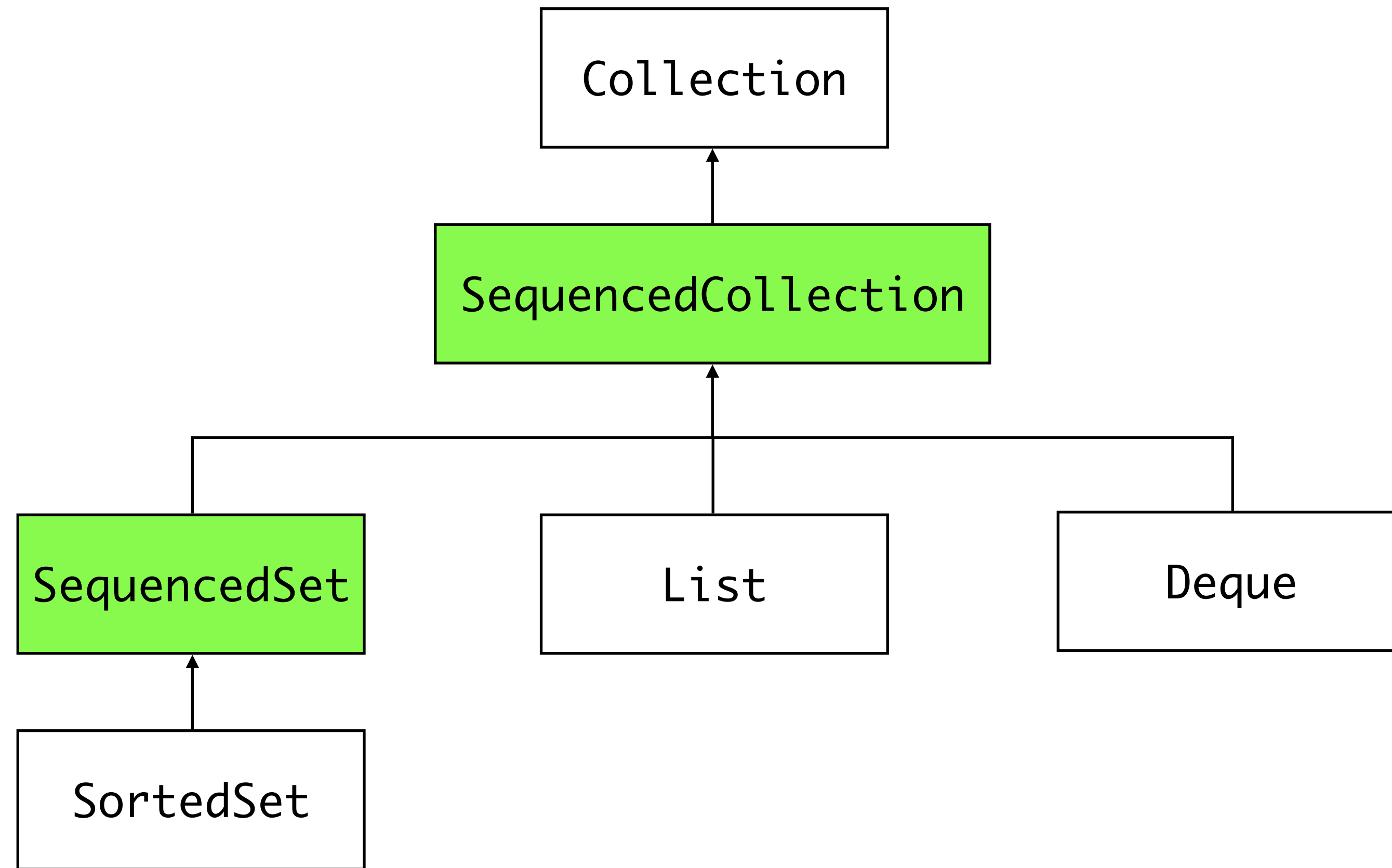

Sequenced Collections & Maps

Collections

- Prior to v21 no uniform way to handle ordered collections:
 - `var first = deque.getFirst();`
 - `var first = list.get(0);`
- New interfaces: `SequencedCollection`, `SequencedSet`, & `SequencedMap`

Sequenced Collections & Maps

Collections



SequencedCollection

Collections

```
interface SequencedCollection<E> extends Collection<E> {  
    SequencedCollection<E> reversed(); // returns a view!  
  
    // methods promoted from Deque  
    void addFirst();  
    void addLast();  
    E getFirst();  
    E getLast();  
    E removeFirst();  
    E removeLast();  
}
```


SequencedSet

Collections

```
interface SequencedSet<E>  
    extends Set<E>, SequencedCollection<E> {  
  
    SequencedSet<E> reversed(); // returns a view!  
}
```

SequencedMap

Collections

```
interface SequencedMap<K, V> extends Map<K, V> {  
    // all return a view!  
    SequencedMap<K, V> reversed();  
    SequencedSet<K> sequencedKeySet();  
    SequencedCollection<V> sequencedValues();  
    SequencedSet<Map.Entry<K, V>> sequencedEntrySet();  
    ...  
}
```


SequencedMap

Collections

```
interface SequencedMap<K, V> extends Map<K, V> {  
    ...  
    V putFirst(K, V);  
    V putLast(K, V);  
  
    // methods promoted from NavigableMap  
    Map.Entry<K, V> firstEntry();  
    Map.Entry<K, V> lastEntry();  
    Map.Entry<K, V> pollFirstEntry();  
    Map.Entry<K, V> pollLastEntry();  
}
```